INDIVIDUAL PROJECT REPORT

Review 2 — Design, Development & Progress (14/9/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role
Anvay Sharma	24BCE0824	_____	Edge gateway and edge intelligence

Team: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417). **Repository:** https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Summary

Since Review 1 I built the edge gateway of our prototype: it receives every sensor packet, classifies it as CRITICAL or NORMAL against cyclone thresholds, forwards critical packets first through a priority queue, and throttles normal traffic while a cyclone alert is active. This report covers the gateway's design, the working code, my design choices, feasibility, the problems I found by testing it, and my contribution.

1. Detailed Design

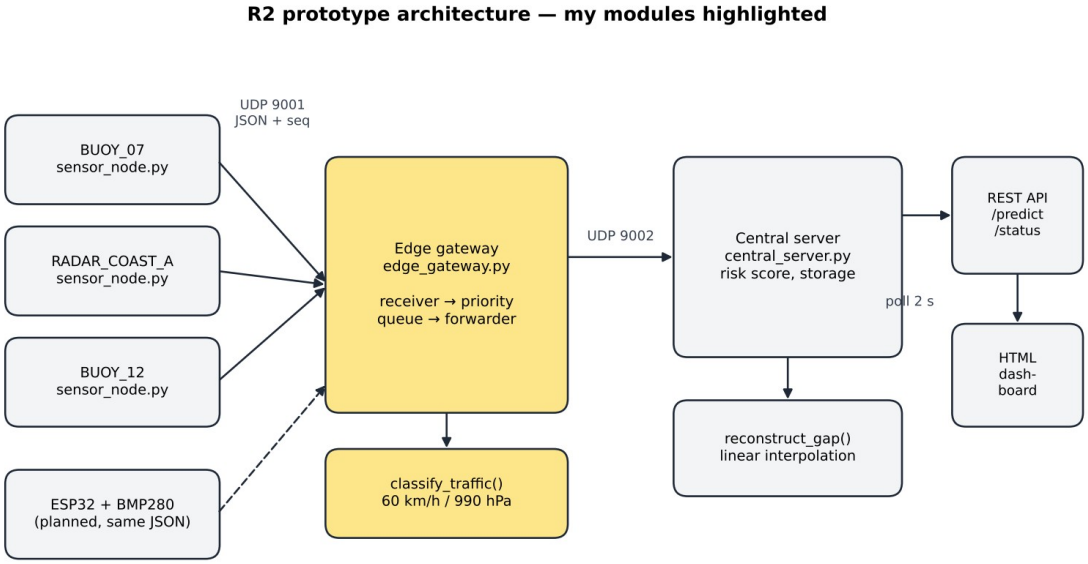


Figure 1: Architecture of the R2 prototype; my modules (gateway and classification rule) are highlighted

Figure 1 shows where the gateway sits between the sensors and the server. It has two threads (Figure 2). The **receiver** reads UDP port 9001, decodes the record, classifies it and pushes (priority, order, record) into a priority queue. The **forwarder** takes the highest-priority record and sends it to the server on port 9002; while an alert is active, each NORMAL packet waits 0.5 s first. The alert switches on with any CRITICAL packet and clears after 10 s without one.

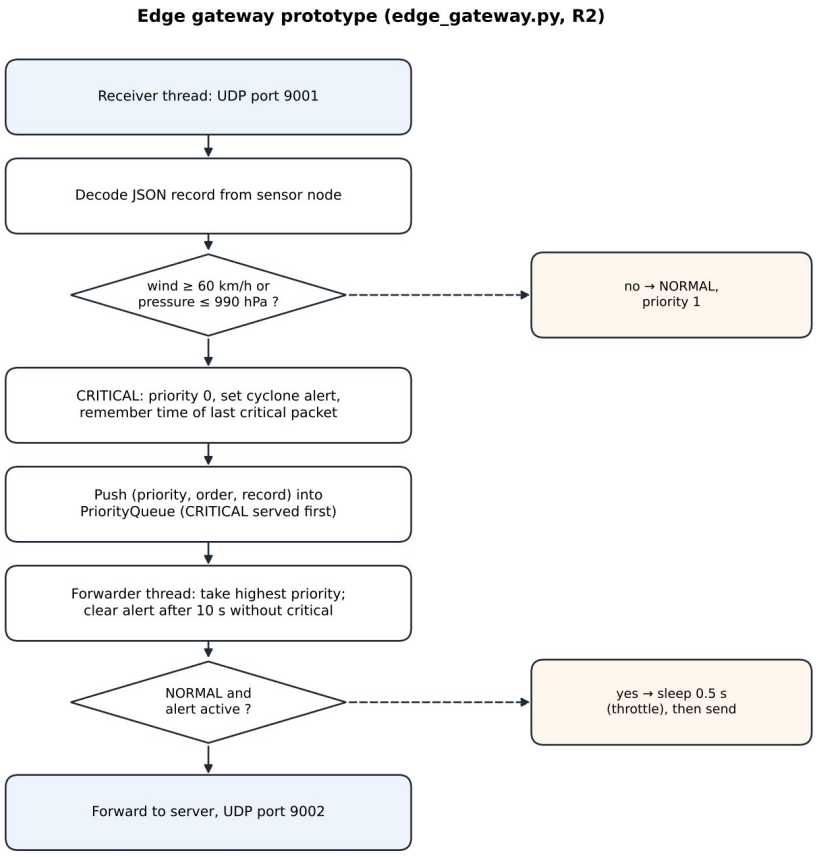


Figure 2: Flowchart of my edge gateway (prototype)

Table 1: Labelled parameters of my module

Parameter	Value	Meaning
SENSOR_TO_GATEWAY_PORT	9001 (UDP)	Input from sensor nodes
GATEWAY_TO_SERVER_PORT	9002 (UDP)	Output to the central server
WIND_CRITICAL_THRESHOLD	60 km/h	CRITICAL if wind ≥ this
PRESSURE_CRITICAL_THRESHOLD	990 hPa	CRITICAL if pressure ≤ this
Priority	0 = CRITICAL, 1 = NORMAL	Lower value served first; insertion order breaks ties
ALERT_HOLD_SECONDS	10 s	Alert stays on this long after the last CRITICAL packet
Throttle	0.5 s per NORMAL packet	Applied only while the alert is active

2. Actual Working Progress since Review 1

- **Code committed:** `edge_gateway.py` (101 lines) and the `classify_traffic()` rule in `common.py`, commit `82bcb8a`.
- **Running:** the gateway receives from three nodes and forwards in priority order; the log shows the alert switching on and NORMAL packets being throttled.
- **Studied for the next phase:** trend-based ML classification and satellite feature extraction (literature from Review 1).

```
def classify_traffic(record):
    if record["wind"] >= WIND_CRITICAL_THRESHOLD or record["pressure"] <=
PRESSURE_CRITICAL_THRESHOLD:
        return TRAFFIC_CRITICAL
    return TRAFFIC_NORMAL

# forwarder thread
_priority, _order, record = pq.get()
if record["traffic_class"] != TRAFFIC_CRITICAL and cyclone_alert.is_set():
    time.sleep(0.5)            # reserve bandwidth for critical traffic
sock.sendto(encode(record), (HOST, GATEWAY_TO_SERVER_PORT))
```

The gateway evidence below comes from a 60-second re-run of the unmodified Review 2 code (commit `82bcb8a`) with three nodes; `gateway.log` and `crash_gw.log` are kept in `docs/reports/evidence/r2_prototype/`. In the team's first 12-second test the server received 66 packets through my gateway, 2 of them labelled CRITICAL.

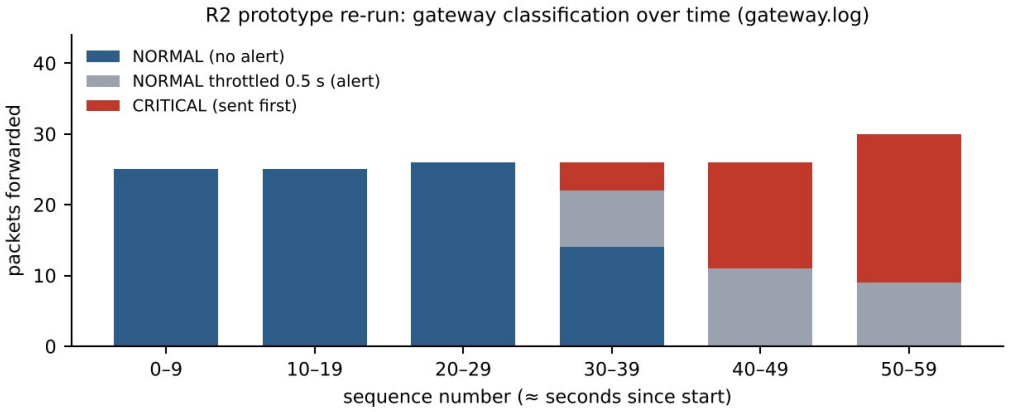


Figure 3: Classification over time in the re-run (from my gateway's log)

In the re-run the gateway forwarded 160 packets: 42 CRITICAL packets were sent immediately, and 28 NORMAL packets were throttled during the alert.

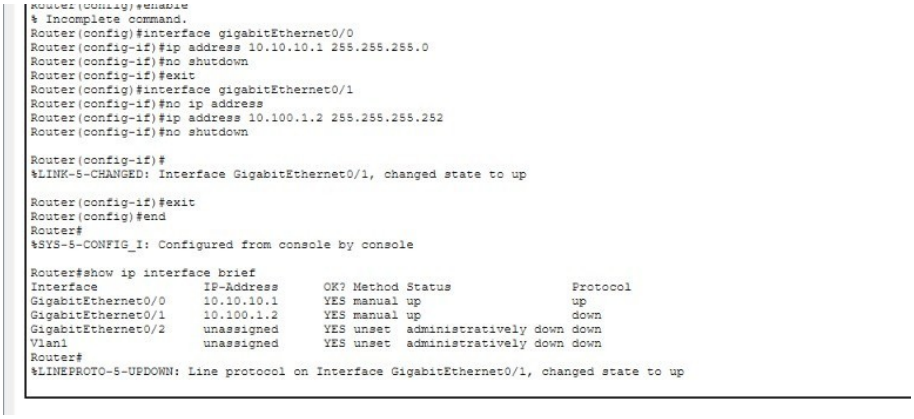


Figure 4: Ocean-side router of the team's Packet Tracer topology (joint work); this is the edge of the network where my gateway sits

Completion: about 40 % of my planned scope. **Done:** gateway pipeline, threshold classification, priority forwarding, alert mode. **Remaining:** input validation, an ML early-warning classifier, satellite image feature extraction, an LSTM for missing data, and a proper evaluation.

Table 2: Test runs performed on my module

Test run	Setup	Observation
Team test, 12 s, 2 nodes	Gateway between sensors and server	2 packets classified CRITICAL once wind > 60
Re-run, 60 s, 3 nodes	Prototype commit 82bcb8a	160 forwarded; 42 CRITICAL sent first; 28 NORMAL throttled
Robustness test	One non-JSON datagram to port 9001	Receiver thread died; later packets ignored
Alert-hold check	Code review of the forwarder loop	Alert can be stale after a silent period

3. Justification of Design Choices

Table 3: Comparison of alternatives for my module (cost, accuracy, availability, power, ease of use)

Choice	Option	Cost	Accuracy / fit	Availability	Power	Ease of use	Decision
Classifier	**Threshold rule**	₹0	Exact on breaches; no early warning	Ready in common.py	Negligible	Very easy	Chosen now
	Logistic regression on trends	₹0	Can warn before a breach	NumPy (free)	Very low	Easy	Next phase
	Deep network	GPU or paid cloud	Possibly best	Needs labelled data we lack	High	Hard	Rejected
Forwarding	FIFO queue	₹0	Critical waits behind others	Standard library	Low	Easiest	Rejected
	Fixed cap per class	₹0	Wastes capacity when idle	Standard library	Low	Easy	Rejected
	Priority queue	₹0	Critical first; adapts instantly	queue.PriorityQueue	Low	Easy	Chosen
Edge device	**Laptop**	Owned	Fast enough (< 1 % CPU)	Available now	≈30–60 W	Easy	Chosen
	Raspberry Pi 4	≈₹5,000	Fast enough	College lab	≈3–6 W	Moderate	Optional

4. Updated Feasibility

Table 4: Economic picture for my part

Aspect	Review 1 estimate	Now (Review 2)	Reason
Software cost	₹0	₹0	Python standard library only
Edge hardware	Laptop or Raspberry Pi	Laptop; Pi optional (lab)	Gateway uses < 1 % CPU
ML compute	Possibly GPU	CPU only	Lightweight models chosen to fit the edge

Social: earlier recognition of danger directly benefits people who must evacuate. **Environmental:** choosing small CPU models instead of GPU training keeps energy use low, and prioritising data avoids wasteful retransmissions on remote links.

5. Problems Encountered and How I Adapted

Table 5: Problems in my module

Problem	How I diagnosed it	What I changed	Result
A malformed packet stopped the gateway	Sent a non-JSON datagram while testing: the receiver thread raised an exception in decode() and died; later valid packets were ignored	Added a validation step to the design (check fields and ranges, count and drop bad packets)	To be implemented next; logged as the top fix
NORMAL packets delayed even on an idle link	Log showed 28 NORMAL packets each waiting 0.5 s during the alert although the queue was empty; a fixed sleep ignores real link capacity	Plan a capacity-based scheduler (token bucket) instead of a fixed sleep	Behaviour understood; redesign scheduled
Alert state can be stale	Code review: the alert is cleared only before the forwarder blocks on the queue, so a NORMAL packet arriving after a long silence is still throttled	Re-check the alert after every dequeue in the redesign	Identified before it affected a demo

Plan from Review 2 to Review 3

Week	Planned work
15–21 Sep	Input validation and capacity-based forwarding in the gateway
22–28 Sep	Trend-based ML classifier (logistic regression) with the rule as safety floor
29 Sep–5 Oct	Satellite feature extraction; LSTM implementation in NumPy
6–9 Oct	Evaluation harness, results tables, trained models in the repository

6. Individual Contribution Evidence

Table 6: Division of work

Tasks I did	Tasks done by teammates
Edge gateway: receiver and forwarder threads, priority queue	Laksh: central server, REST API, dashboard, startup fix
CRITICAL / NORMAL rule (classify_traffic) and alert mode	Sreehari: sensor-node simulator and gap interpolation
Robustness testing of the gateway (malformed input, throttling)	Team: shared record format, Packet Tracer topology

- Code: `edge_gateway.py` and `classify_traffic()` in `common.py`, commit `82bcb8a`

(https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).

- Logs: `gateway.log` and `crash_gw.log` in `docs/reports/evidence/r2_prototype/`.

> *[Insert: dated screenshot of edge_gateway.py printing CRITICAL / NORMAL lines while I test it, plus a photo of me at the laptop during the test. Commits from my own GitHub account will be added from R3.]*

R2 Video Plan

Segment	What I show
Design walkthrough (2 min)	Face + screen; the architecture and my gateway flowchart
Live progress with my own work on screen (2 min)	Face + screen; gateway log classifying packets live
One problem and its solution (1 min)	Face + screen; the crash on bad input and the 0.5 s throttle problem
Updated cost and my contribution (1 min)	Face + screen; comparison table and division of work